# 1. Imports

In [1]:
```
import requests
```

# 2. Version

In [2]:
```
print(requests.__version__)
```

```
2.32.5
```

# 3. Working with `GET` Method

## 3.1

- Send a request to GitHub API and print the response
- Print the status code and the contents
- GitHub API: `https://api.github.com`

In [3]:
```
uri = "https://api.github.com"
response = requests.get(uri)
```

In [4]:
```
response
```

Out[4]:
```
<Response [200]>
```

In [5]:
```
response.status_code
```

Out[5]:
```
200
```

In [6]:
```
response.content
```

Out[6]:
b'{\n  "current_user_url": "https://api.github.com/user",\n  "current_user_auth
orizations_html_url": "https://github.com/settings/connections/applications{/cl
ient_id}",\n  "authorizations_url": "https://api.github.com/authorizations",\n
"code_search_url": "https://api.github.com/search/code?q={query}{&page,per_pag
e,sort,order}",\n  "commit_search_url": "https://api.github.com/search/commits?
q={query}{&page,per_page,sort,order}",\n  "emails_url": "https://api.github.co
m/user/emails",\n  "emojis_url": "https://api.github.com/emojis",\n  "events_ur
l": "https://api.github.com/events",\n  "feeds_url": "https://api.github.com/fe
eds",\n  "followers_url": "https://api.github.com/user/followers",\n  "followin
g_url": "https://api.github.com/user/following{/target}",\n  "gists_url": "http
s://api.github.com/gists{/gist_id}",\n  "hub_url": "https://api.github.com/hu
b",\n  "issue_search_url": "https://api.github.com/search/issues?q={query}{&pag
e,per_page,sort,order}",\n  "issues_url": "https://api.github.com/issues",\n
"keys_url": "https://api.github.com/user/keys",\n  "label_search_url": "http
s://api.github.com/search/labels?q={query}&repository_id={repository_id}{&page,
per_page}",\n  "notifications_url": "https://api.github.com/notifications",\n
"organization_url": "https://api.github.com/orgs/{org}",\n  "organization_repos
itories_url": "https://api.github.com/orgs/{org}/repos{?type,page,per_page,sor
t}",\n  "organization_teams_url": "https://api.github.com/orgs/{org}/teams",\n
"public_gists_url": "https://api.github.com/gists/public",\n  "rate_limit_url":
"https://api.github.com/rate_limit",\n  "repository_url": "https://api.github.c
om/repos/{owner}/{repo}",\n  "repository_search_url": "https://api.github.com/s
earch/repositories?q={query}{&page,per_page,sort,order}",\n  "current_user_repo
sitories_url": "https://api.github.com/user/repos{?type,page,per_page,sort}",\n
"starred_url": "https://api.github.com/user/starred{/owner}{/repo}",\n  "starre
d_gists_url": "https://api.github.com/gists/starred",\n  "topic_search_url": "h
ttps://api.github.com/search/topics?q={query}{&page,per_page}",\n  "user_url":
"https://api.github.com/users/{user}",\n  "user_organizations_url": "https://ap
i.github.com/user/orgs",\n  "user_repositories_url": "https://api.github.com/us
ers/{user}/repos{?type,page,per_page,sort}",\n  "user_search_url": "https://ap
i.github.com/search/users?q={query}{&page,per_page,sort,order}"\n}\n'

In [7]:
```
response.content[:1000]
```

Out[7]:
b'{\n  "current_user_url": "https://api.github.com/user",\n  "current_user_auth
orizations_html_url": "https://github.com/settings/connections/applications{/cl
ient_id}",\n  "authorizations_url": "https://api.github.com/authorizations",\n
"code_search_url": "https://api.github.com/search/code?q={query}{&page,per_pag
e,sort,order}",\n  "commit_search_url": "https://api.github.com/search/commits?
q={query}{&page,per_page,sort,order}",\n  "emails_url": "https://api.github.co
m/user/emails",\n  "emojis_url": "https://api.github.com/emojis",\n  "events_ur
l": "https://api.github.com/events",\n  "feeds_url": "https://api.github.com/fe
eds",\n  "followers_url": "https://api.github.com/user/followers",\n  "followin
g_url": "https://api.github.com/user/following{/target}",\n  "gists_url": "http
s://api.github.com/gists{/gist_id}",\n  "hub_url": "https://api.github.com/hu
b",\n  "issue_search_url": "https://api.github.com/search/issues?q={query}{&pag
e,per_page,sort,order}",\n  "issues_url": "https://api.github.com/issues'

# 3.2

- We use query parameters in the GET message
- It searches for all GitHub repositories that contain the word `requests`, and the main language used is `python`
- GitHub repository search API: `https://api.github.com/search/repositories`

In [8]:
```
uri = "https://api.github.com/search/repositories"
```

```python
params = {"q": "requests+language:python"}

response = requests.get(uri, params = params)

response
```

Out[8]: <Response [200]>

In [9]:
```python
response.status_code
```

Out[9]: 200

In [10]:
```python
# response.content
response.content[:1000]
```

Out[10]:
b'{"total_count":318,"incomplete_results":false,"items":[{"id":972296490,"node_id":"R_kgDOOfQRKg","name":"calendar-mcp","full_name":"deciduus/calendar-mcp","private":false,"owner":{"login":"deciduus","id":151121987,"node_id":"U_kgDOCQHwQw","avatar_url":"https://avatars.githubusercontent.com/u/151121987?v=4","gravatar_id":"","url":"https://api.github.com/users/deciduus","html_url":"https://github.com/deciduus","followers_url":"https://api.github.com/users/deciduus/followers","following_url":"https://api.github.com/users/deciduus/following{/other_user}","gists_url":"https://api.github.com/users/deciduus/gists{/gist_id}","starred_url":"https://api.github.com/users/deciduus/starred{/owner}{/repo}","subscriptions_url":"https://api.github.com/users/deciduus/subscriptions","organizations_url":"https://api.github.com/users/deciduus/orgs","repos_url":"https://api.github.com/users/deciduus/repos","events_url":"https://api.github.com/users/deciduus/events{/privacy}","received_events_url":"https://a'

In [11]:
```python
response.content[:2000]
```

Out[11]:
b'{"total_count":318,"incomplete_results":false,"items":[{"id":972296490,"node_id":"R_kgDOOfQRKg","name":"calendar-mcp","full_name":"deciduus/calendar-mcp","private":false,"owner":{"login":"deciduus","id":151121987,"node_id":"U_kgDOCQHwQw","avatar_url":"https://avatars.githubusercontent.com/u/151121987?v=4","gravatar_id":"","url":"https://api.github.com/users/deciduus","html_url":"https://github.com/deciduus","followers_url":"https://api.github.com/users/deciduus/followers","following_url":"https://api.github.com/users/deciduus/following{/other_user}","gists_url":"https://api.github.com/users/deciduus/gists{/gist_id}","starred_url":"https://api.github.com/users/deciduus/starred{/owner}{/repo}","subscriptions_url":"https://api.github.com/users/deciduus/subscriptions","organizations_url":"https://api.github.com/users/deciduus/orgs","repos_url":"https://api.github.com/users/deciduus/repos","events_url":"https://api.github.com/users/deciduus/events{/privacy}","received_events_url":"https://api.github.com/users/deciduus/received_events","type":"User","user_view_type":"public","site_admin":false},"html_url":"https://github.com/deciduus/calendar-mcp","description":"This project implements a Python-based MCP (Model Context Protocol) server that acts as an interface between Large Language Models (LLMs) and the Google Calendar API. It enables LLMs to perform calendar operations via natural language requests.","fork":false,"url":"https://api.github.com/repos/deciduus/calendar-mcp","forks_url":"https://api.github.com/repos/deciduus/calendar-mcp/forks","keys_url":"https://api.github.com/repos/deciduus/calendar-mcp/keys{/key_id}","collaborators_url":"https://api.github.com/repos/deciduus/calendar-mcp/collaborators{/collaborator}","teams_url":"https://api.github.com/repos/deciduus/calendar-mcp/teams","hooks_url":"https://api.github.com/repos/deciduus/calendar-mcp/hooks","issue_events_url":"https://api.github.com/repos/deciduus/calendar-mcp/issues/events{/number}","events_url":"https'

# 4. Working with `POST` Method

## 4.1

- We send some data to a test server
- server address: `https://httpbin.org/post`

In [12]:
```
uri = "https://httpbin.org/post"

data = {
    "username": "Emilia Clarke",
    "password": "Daenerys Targaryen"
}

response = requests.post(uri, data = data)
```

In [13]:
```
response.status_code
```

Out[13]:
```
200
```

In [14]:
```
response.json()
```

Out[14]:
```
{'args': {},
 'data': '',
 'files': {},
 'form': {'password': 'Daenerys Targaryen', 'username': 'Emilia Clarke'},
 'headers': {'Accept': '*/*',
  'Accept-Encoding': 'gzip, deflate, zstd',
  'Content-Length': '50',
  'Content-Type': 'application/x-www-form-urlencoded',
  'Host': 'httpbin.org',
  'User-Agent': 'python-requests/2.32.5',
  'X-Amzn-Trace-Id': 'Root=1-69b2195e-2c0ba3d94eb750ca08631e23'},
 'json': None,
 'origin': '157.50.77.70',
 'url': 'https://httpbin.org/post'}
```

- By default, the data we send using requests is transmitted as form data.
- In headers, the content type value `x-www-form-urlencoded` indicates the data format sent in the request.

In [15]:
```
response.json()['headers']
```

Out[15]:
```
{'Accept': '*/*',
 'Accept-Encoding': 'gzip, deflate, zstd',
 'Content-Length': '50',
 'Content-Type': 'application/x-www-form-urlencoded',
 'Host': 'httpbin.org',
 'User-Agent': 'python-requests/2.32.5',
 'X-Amzn-Trace-Id': 'Root=1-69b2195e-2c0ba3d94eb750ca08631e23'}
```

In [16]:
```
response.json()['headers']['Content-Type']
```

Out[16]: 'application/x-www-form-urlencoded'

In [17]:
```
response.json()['headers']['Content-Type'].split("/")[-1]
```

Out[17]: 'x-www-form-urlencoded'

In [18]:
```
# function that gives us data format

def get_data_format(response_got):
    json_response = response_got.json()
    data_format = json_response["headers"]["Content-Type"].split("/")[-1]
    print(f"Response data format: {data_format}")
```

In [19]:
```
get_data_format(response)
```

```
Response data format: x-www-form-urlencoded
```

## 4.2

- APIs usually expect data in JSON format
- We send the data in JSON format using `POST` message

In [20]:
```
uri
```

Out[20]: 'https://httpbin.org/post'

In [21]:
```
data
```

Out[21]: {'username': 'Emilia Clarke', 'password': 'Daenerys Targaryen'}

In [22]:
```
response2 = requests.post(uri, json = data)
```

In [23]:
```
response2.status_code
```

Out[23]: 200

In [24]:
```
response2.json()
```

Out[24]:
```
{'args': {},
 'data': '{"username": "Emilia Clarke", "password": "Daenerys Targaryen"}',
 'files': {},
 'form': {},
 'headers': {'Accept': '*/*',
  'Accept-Encoding': 'gzip, deflate, zstd',
  'Content-Length': '63',
  'Content-Type': 'application/json',
  'Host': 'httpbin.org',
  'User-Agent': 'python-requests/2.32.5',
  'X-Amzn-Trace-Id': 'Root=1-69b2198a-488801ff66d681bd74a59b60'},
 'json': {'password': 'Daenerys Targaryen', 'username': 'Emilia Clarke'},
 'origin': '157.50.77.70',
 'url': 'https://httpbin.org/post'}
```

- Ensure you compare the results of `response.json()` and `response2.json()`
- Particularly focusing on attributes like `data`, `form`, and `json`

In [25]:
```
get_data_format(response)
```

```
Response data format: x-www-form-urlencoded
```

In [26]:
```
get_data_format(response2)
```

```
Response data format: json
```

# 5. Working with `PUT` Method

## 5.1

- We use `PUT` method to update data
- Address: `https://httpbin.org/put`

In [27]:
```
uri = "https://httpbin.org/put"

data = {
    "param1": "value1"
}

response = requests.put(uri, data = data)
```

In [28]:
```
response.status_code
```

Out[28]:
```
200
```

In [29]:
```
response.json()
```

Out[29]:
```
{'args': {},
 'data': '',
 'files': {},
 'form': {'param1': 'value1'},
 'headers': {'Accept': '*/*',
  'Accept-Encoding': 'gzip, deflate, zstd',
  'Content-Length': '13',
  'Content-Type': 'application/x-www-form-urlencoded',
  'Host': 'httpbin.org',
  'User-Agent': 'python-requests/2.32.5',
  'X-Amzn-Trace-Id': 'Root=1-69b21a70-397874dc54b3dd71197c1df4'},
 'json': None,
 'origin': '157.50.77.70',
 'url': 'https://httpbin.org/put'}
```

# 6. Working with `DELETE` Method

## 6.1

- We use `DELETE` method to delete a resource
- address: `https://httpbin.org/delete`

In [30]:
```
uri = "https://httpbin.org/delete"

response = requests.delete(uri)
```

In [31]:
```
response.status_code
```

Out[31]:
```
200
```

In [32]:
```
response.json()
```

Out[32]:
```
{'args': {},
 'data': '',
 'files': {},
 'form': {},
 'headers': {'Accept': '*/*',
  'Accept-Encoding': 'gzip, deflate, zstd',
  'Content-Length': '0',
  'Host': 'httpbin.org',
  'User-Agent': 'python-requests/2.32.5',
  'X-Amzn-Trace-Id': 'Root=1-69b21a7a-47d3bc3850f7f90d4a10efb1'},
 'json': None,
 'origin': '157.50.77.70',
 'url': 'https://httpbin.org/delete'}
```

# 7. Working with HTTP Headers

- We use a GET request to work with headers
- Url: `https://httpbin.org/headers`

In [33]:
```
headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)',
    'Accept': 'application/json',
        'Authorization': 'Bearer YOUR_ACCESS_TOKEN',
        'Content-Type': 'application/json',
        'X-Custom-Header': 'CustomValue',
}

uri = "https://httpbin.org/headers"
```

In [34]:
```
response = requests.get(uri, headers = headers)
```

In [35]:
```
response.status_code
```

Out[35]:
```
200
```

In [36]:
```
response.headers
```

Out[36]:
```
{'Date': 'Thu, 12 Mar 2026 01:48:08 GMT', 'Content-Type': 'application/json',
'Content-Length': '393', 'Connection': 'keep-alive', 'Server': 'gunicorn/19.9.
0', 'Access-Control-Allow-Origin': '*', 'Access-Control-Allow-Credentials': 'tr
ue'}
```

In [37]:
```
type(response.headers)
```

Out[37]:
```
requests.structures.CaseInsensitiveDict
```

In [38]:
```
dict(response.headers)
```

Out[38]:
```
{'Date': 'Thu, 12 Mar 2026 01:48:08 GMT',
 'Content-Type': 'application/json',
 'Content-Length': '393',
 'Connection': 'keep-alive',
 'Server': 'gunicorn/19.9.0',
 'Access-Control-Allow-Origin': '*',
 'Access-Control-Allow-Credentials': 'true'}
```

In [39]:
```
response.json()
```

Out[39]:
```
{'headers': {'Accept': 'application/json',
  'Accept-Encoding': 'gzip, deflate, zstd',
  'Authorization': 'Bearer YOUR_ACCESS_TOKEN',
  'Content-Type': 'application/json',
  'Host': 'httpbin.org',
  'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)',
  'X-Amzn-Trace-Id': 'Root=1-69b21b58-0071df604dfc2c8a7bc36f47',
  'X-Custom-Header': 'CustomValue'}}
```

In [40]:
```
response.json()["headers"]
```

Out[40]:
```
{'Accept': 'application/json',
 'Accept-Encoding': 'gzip, deflate, zstd',
 'Authorization': 'Bearer YOUR_ACCESS_TOKEN',
 'Content-Type': 'application/json',
 'Host': 'httpbin.org',
 'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)',
 'X-Amzn-Trace-Id': 'Root=1-69b21b58-0071df604dfc2c8a7bc36f47',
 'X-Custom-Header': 'CustomValue'}
```

In [41]:
```
for header, value in response.json()["headers"].items():
    print(f"{header:<16}: {value}")
```

```
Accept          : application/json
Accept-Encoding : gzip, deflate, zstd
Authorization   : Bearer YOUR_ACCESS_TOKEN
Content-Type    : application/json
Host            : httpbin.org
User-Agent      : Mozilla/5.0 (Windows NT 10.0; Win64; x64)
X-Amzn-Trace-Id : Root=1-69b21b58-0071df604dfc2c8a7bc36f47
X-Custom-Header : CustomValue
```

# 8. Working with Response Object

- Address: `https://api.github.com`
- The most common attributes of the response object are:
  - `status_code` : shows the HTTP status code of the request
  - `text` : shows the content of the response as a string
  - `content` : shows the content of the response as binary data
  - `headers` : shows all the response headers
  - `json()` : parse the server's response as JSON

In [42]:
```
uri = "https://api.github.com"
```

```
response = requests.get(uri)
```

In [43]:
```
response
```

Out[43]:
```
<Response [200]>
```

In [44]:
```
response.status_code
```

Out[44]:
```
200
```

In [45]:
```
type(response.status_code)
```

Out[45]:
```
int
```

In [46]:
```
if response.status_code == 200:
    print("Successful Request!")
else:
    print("Unsuccessful Request!")
```

```
Successful Request!
```

In [47]:
```
response.text
```

Out[47]:

```
'{"current_user_url":"https://api.github.com/user","current_user_authorizations
_html_url":"https://github.com/settings/connections/applications{/client_i
d}","authorizations_url":"https://api.github.com/authorizations","code_search_u
rl":"https://api.github.com/search/code?q={query}{&page,per_page,sort,orde
r}","commit_search_url":"https://api.github.com/search/commits?q={query}{&page,
per_page,sort,order}","emails_url":"https://api.github.com/user/emails","emojis
_url":"https://api.github.com/emojis","events_url":"https://api.github.com/even
ts","feeds_url":"https://api.github.com/feeds","followers_url":"https://api.git
hub.com/user/followers","following_url":"https://api.github.com/user/following
{/target}","gists_url":"https://api.github.com/gists{/gist_id}","hub_url":"http
s://api.github.com/hub","issue_search_url":"https://api.github.com/search/issue
s?q={query}{&page,per_page,sort,order}","issues_url":"https://api.github.com/is
sues","keys_url":"https://api.github.com/user/keys","label_search_url":"http
s://api.github.com/search/labels?q={query}&repository_id={repository_id}{&page,
per_page}","notifications_url":"https://api.github.com/notifications","organiza
tion_url":"https://api.github.com/orgs/{org}","organization_repositories_ur
l":"https://api.github.com/orgs/{org}/repos{?type,page,per_page,sort}","organiz
ation_teams_url":"https://api.github.com/orgs/{org}/teams","public_gists_ur
l":"https://api.github.com/gists/public","rate_limit_url":"https://api.github.c
om/rate_limit","repository_url":"https://api.github.com/repos/{owner}/{rep
o}","repository_search_url":"https://api.github.com/search/repositories?q={quer
y}{&page,per_page,sort,order}","current_user_repositories_url":"https://api.git
hub.com/user/repos{?type,page,per_page,sort}","starred_url":"https://api.githu
b.com/user/starred{/owner}{/repo}","starred_gists_url":"https://api.github.com/
gists/starred","topic_search_url":"https://api.github.com/search/topics?q={quer
y}{&page,per_page}","user_url":"https://api.github.com/users/{user}","user_orga
nizations_url":"https://api.github.com/user/orgs","user_repositories_url":"http
s://api.github.com/users/{user}/repos{?type,page,per_page,sort}","user_search_u
rl":"https://api.github.com/search/users?q={query}{&page,per_page,sort,orde
r}"}'
```


In [48]:
```
type(response.text)
```

Out[48]:
```
str
```

In [49]:
```
response.content
```

Out[49]:
b'{"current_user_url":"https://api.github.com/user","current_user_authorization
s_html_url":"https://github.com/settings/connections/applications{/client_i
d}","authorizations_url":"https://api.github.com/authorizations","code_search_u
rl":"https://api.github.com/search/code?q={query}{&page,per_page,sort,orde
r}","commit_search_url":"https://api.github.com/search/commits?q={query}{&page,
per_page,sort,order}","emails_url":"https://api.github.com/user/emails","emojis
_url":"https://api.github.com/emojis","events_url":"https://api.github.com/even
ts","feeds_url":"https://api.github.com/feeds","followers_url":"https://api.git
hub.com/user/followers","following_url":"https://api.github.com/user/following
{/target}","gists_url":"https://api.github.com/gists{/gist_id}","hub_url":"http
s://api.github.com/hub","issue_search_url":"https://api.github.com/search/issue
s?q={query}{&page,per_page,sort,order}","issues_url":"https://api.github.com/is
sues","keys_url":"https://api.github.com/user/keys","label_search_url":"http
s://api.github.com/search/labels?q={query}&repository_id={repository_id}{&page,
per_page}","notifications_url":"https://api.github.com/notifications","organiza
tion_url":"https://api.github.com/orgs/{org}","organization_repositories_ur
l":"https://api.github.com/orgs/{org}/repos{?type,page,per_page,sort}","organiz
ation_teams_url":"https://api.github.com/orgs/{org}/teams","public_gists_ur
l":"https://api.github.com/gists/public","rate_limit_url":"https://api.github.c
om/rate_limit","repository_url":"https://api.github.com/repos/{owner}/{rep
o}","repository_search_url":"https://api.github.com/search/repositories?q={quer
y}{&page,per_page,sort,order}","current_user_repositories_url":"https://api.git
hub.com/user/repos{?type,page,per_page,sort}","starred_url":"https://api.githu
b.com/user/starred{/owner}{/repo}","starred_gists_url":"https://api.github.com/
gists/starred","topic_search_url":"https://api.github.com/search/topics?q={quer
y}{&page,per_page}","user_url":"https://api.github.com/users/{user}","user_orga
nizations_url":"https://api.github.com/user/orgs","user_repositories_url":"http
s://api.github.com/users/{user}/repos{?type,page,per_page,sort}","user_search_u
rl":"https://api.github.com/search/users?q={query}{&page,per_page,sort,orde
r}"}'

In [50]:
```
type(response.content)
```

Out[50]:
```
bytes
```

In [51]:
```
response.headers
```

Out[51]:
{'Date': 'Thu, 12 Mar 2026 01:54:26 GMT', 'Content-Type': 'application/json; ch
arset=utf-8', 'Cache-Control': 'public, max-age=60, s-maxage=60', 'Vary': 'Acce
pt,Accept-Encoding, Accept, X-Requested-With', 'ETag': 'W/"4f825cc84e1c733059d4
6e76e6df9db557ae5254f9625dfe8e1b09499c449438"', 'X-GitHub-Media-Type': 'github.
v3; format=json', 'x-github-api-version-selected': '2022-11-28', 'Access-Contro
l-Expose-Headers': 'ETag, Link, Location, Retry-After, X-GitHub-OTP, X-RateLimi
t-Limit, X-RateLimit-Remaining, X-RateLimit-Used, X-RateLimit-Resource, X-RateL
imit-Reset, X-OAuth-Scopes, X-Accepted-OAuth-Scopes, X-Poll-Interval, X-GitHub-
Media-Type, X-GitHub-SSO, X-GitHub-Request-Id, Deprecation, Sunset', 'Access-Co
ntrol-Allow-Origin': '*', 'Strict-Transport-Security': 'max-age=31536000; inclu
deSubdomains; preload', 'X-Frame-Options': 'deny', 'X-Content-Type-Options': 'n
osniff', 'X-XSS-Protection': '0', 'Referrer-Policy': 'origin-when-cross-origin,
strict-origin-when-cross-origin', 'Content-Security-Policy': "default-src 'non
e'", 'Content-Encoding': 'gzip', 'Server': 'github.com', 'Accept-Ranges': 'byte
s', 'X-RateLimit-Limit': '60', 'X-RateLimit-Remaining': '58', 'X-RateLimit-Use
d': '2', 'X-RateLimit-Resource': 'core', 'X-RateLimit-Reset': '1773282519', 'Co
ntent-Length': '510', 'X-GitHub-Request-Id': 'C7DF:0BA9:CF189:12C46E:69B21CD2'}

In [52]:
```
type(response.headers)
```

Out[52]:
```
requests.structures.CaseInsensitiveDict
```

In [53]:
```
dict(response.headers)
```

Out[53]:
```
{'Date': 'Thu, 12 Mar 2026 01:54:26 GMT',
 'Content-Type': 'application/json; charset=utf-8',
 'Cache-Control': 'public, max-age=60, s-maxage=60',
 'Vary': 'Accept,Accept-Encoding, Accept, X-Requested-With',
 'ETag': 'W/"4f825cc84e1c733059d46e76e6df9db557ae5254f9625dfe8e1b09499c44943
8"',
 'X-GitHub-Media-Type': 'github.v3; format=json',
 'x-github-api-version-selected': '2022-11-28',
 'Access-Control-Expose-Headers': 'ETag, Link, Location, Retry-After, X-GitHub-
OTP, X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Used, X-RateLimit-Re
source, X-RateLimit-Reset, X-OAuth-Scopes, X-Accepted-OAuth-Scopes, X-Poll-Inte
rval, X-GitHub-Media-Type, X-GitHub-SSO, X-GitHub-Request-Id, Deprecation, Suns
et',
 'Access-Control-Allow-Origin': '*',
 'Strict-Transport-Security': 'max-age=31536000; includeSubdomains; preload',
 'X-Frame-Options': 'deny',
 'X-Content-Type-Options': 'nosniff',
 'X-XSS-Protection': '0',
 'Referrer-Policy': 'origin-when-cross-origin, strict-origin-when-cross-origi
n',
 'Content-Security-Policy': "default-src 'none'",
 'Content-Encoding': 'gzip',
 'Server': 'github.com',
 'Accept-Ranges': 'bytes',
 'X-RateLimit-Limit': '60',
 'X-RateLimit-Remaining': '58',
 'X-RateLimit-Used': '2',
 'X-RateLimit-Resource': 'core',
 'X-RateLimit-Reset': '1773282519',
 'Content-Length': '510',
 'X-GitHub-Request-Id': 'C7DF:0BA9:CF189:12C46E:69B21CD2'}
```

In [54]:
```
for header, value in dict(response.headers).items():
    print(f"{header:<35}: {value}")
```

```
Date                                : Thu, 12 Mar 2026 01:54:26 GMT
Content-Type                        : application/json; charset=utf-8
Cache-Control                       : public, max-age=60, s-maxage=60
Vary                                : Accept,Accept-Encoding, Accept, X-Requested-
With
ETag                                : W/"4f825cc84e1c733059d46e76e6df9db557ae5254f
9625dfe8e1b09499c449438"
X-GitHub-Media-Type                 : github.v3; format=json
x-github-api-version-selected       : 2022-11-28
Access-Control-Expose-Headers       : ETag, Link, Location, Retry-After, X-GitHub-
OTP, X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Used, X-RateLimit-Reso
urce, X-RateLimit-Reset, X-OAuth-Scopes, X-Accepted-OAuth-Scopes, X-Poll-Interva
l, X-GitHub-Media-Type, X-GitHub-SSO, X-GitHub-Request-Id, Deprecation, Sunset
Access-Control-Allow-Origin         : *
Strict-Transport-Security           : max-age=31536000; includeSubdomains; preload
X-Frame-Options                     : deny
X-Content-Type-Options              : nosniff
X-XSS-Protection                    : 0
Referrer-Policy                     : origin-when-cross-origin, strict-origin-when
-cross-origin
Content-Security-Policy             : default-src 'none'
Content-Encoding                    : gzip
Server                              : github.com
Accept-Ranges                       : bytes
X-RateLimit-Limit                   : 60
X-RateLimit-Remaining               : 58
X-RateLimit-Used                    : 2
X-RateLimit-Resource                : core
X-RateLimit-Reset                   : 1773282519
Content-Length                      : 510
X-GitHub-Request-Id                 : C7DF:0BA9:CF189:12C46E:69B21CD2
```

In [55]:
```
response.json()
```

Out[55]:

```
{'current_user_url': 'https://api.github.com/user',
 'current_user_authorizations_html_url': 'https://github.com/settings/connectio
ns/applications{/client_id}',
 'authorizations_url': 'https://api.github.com/authorizations',
 'code_search_url': 'https://api.github.com/search/code?q={query}{&page,per_pag
e,sort,order}',
 'commit_search_url': 'https://api.github.com/search/commits?q={query}{&page,pe
r_page,sort,order}',
 'emails_url': 'https://api.github.com/user/emails',
 'emojis_url': 'https://api.github.com/emojis',
 'events_url': 'https://api.github.com/events',
 'feeds_url': 'https://api.github.com/feeds',
 'followers_url': 'https://api.github.com/user/followers',
 'following_url': 'https://api.github.com/user/following{/target}',
 'gists_url': 'https://api.github.com/gists{/gist_id}',
 'hub_url': 'https://api.github.com/hub',
 'issue_search_url': 'https://api.github.com/search/issues?q={query}{&page,per_
page,sort,order}',
 'issues_url': 'https://api.github.com/issues',
 'keys_url': 'https://api.github.com/user/keys',
 'label_search_url': 'https://api.github.com/search/labels?q={query}&repository
_id={repository_id}{&page,per_page}',
 'notifications_url': 'https://api.github.com/notifications',
 'organization_url': 'https://api.github.com/orgs/{org}',
 'organization_repositories_url': 'https://api.github.com/orgs/{org}/repos{?typ
e,page,per_page,sort}',
 'organization_teams_url': 'https://api.github.com/orgs/{org}/teams',
 'public_gists_url': 'https://api.github.com/gists/public',
 'rate_limit_url': 'https://api.github.com/rate_limit',
 'repository_url': 'https://api.github.com/repos/{owner}/{repo}',
 'repository_search_url': 'https://api.github.com/search/repositories?q={query}
{&page,per_page,sort,order}',
 'current_user_repositories_url': 'https://api.github.com/user/repos{?type,pag
e,per_page,sort}',
 'starred_url': 'https://api.github.com/user/starred{/owner}{/repo}',
 'starred_gists_url': 'https://api.github.com/gists/starred',
 'topic_search_url': 'https://api.github.com/search/topics?q={query}{&page,per_
page}',
 'user_url': 'https://api.github.com/users/{user}',
 'user_organizations_url': 'https://api.github.com/user/orgs',
 'user_repositories_url': 'https://api.github.com/users/{user}/repos{?type,pag
e,per_page,sort}',
 'user_search_url': 'https://api.github.com/search/users?q={query}{&page,per_pa
ge,sort,order}'}
```


In [56]:
```
dict(response.json())
```

Out[56]:

```
{'current_user_url': 'https://api.github.com/user',
 'current_user_authorizations_html_url': 'https://github.com/settings/connectio
ns/applications{/client_id}',
 'authorizations_url': 'https://api.github.com/authorizations',
 'code_search_url': 'https://api.github.com/search/code?q={query}{&page,per_pag
e,sort,order}',
 'commit_search_url': 'https://api.github.com/search/commits?q={query}{&page,pe
r_page,sort,order}',
 'emails_url': 'https://api.github.com/user/emails',
 'emojis_url': 'https://api.github.com/emojis',
 'events_url': 'https://api.github.com/events',
 'feeds_url': 'https://api.github.com/feeds',
 'followers_url': 'https://api.github.com/user/followers',
 'following_url': 'https://api.github.com/user/following{/target}',
 'gists_url': 'https://api.github.com/gists{/gist_id}',
 'hub_url': 'https://api.github.com/hub',
 'issue_search_url': 'https://api.github.com/search/issues?q={query}{&page,per_
page,sort,order}',
 'issues_url': 'https://api.github.com/issues',
 'keys_url': 'https://api.github.com/user/keys',
 'label_search_url': 'https://api.github.com/search/labels?q={query}&repository
_id={repository_id}{&page,per_page}',
 'notifications_url': 'https://api.github.com/notifications',
 'organization_url': 'https://api.github.com/orgs/{org}',
 'organization_repositories_url': 'https://api.github.com/orgs/{org}/repos{?typ
e,page,per_page,sort}',
 'organization_teams_url': 'https://api.github.com/orgs/{org}/teams',
 'public_gists_url': 'https://api.github.com/gists/public',
 'rate_limit_url': 'https://api.github.com/rate_limit',
 'repository_url': 'https://api.github.com/repos/{owner}/{repo}',
 'repository_search_url': 'https://api.github.com/search/repositories?q={query}
{&page,per_page,sort,order}',
 'current_user_repositories_url': 'https://api.github.com/user/repos{?type,pag
e,per_page,sort}',
 'starred_url': 'https://api.github.com/user/starred{/owner}{/repo}',
 'starred_gists_url': 'https://api.github.com/gists/starred',
 'topic_search_url': 'https://api.github.com/search/topics?q={query}{&page,per_
page}',
 'user_url': 'https://api.github.com/users/{user}',
 'user_organizations_url': 'https://api.github.com/user/orgs',
 'user_repositories_url': 'https://api.github.com/users/{user}/repos{?type,pag
e,per_page,sort}',
 'user_search_url': 'https://api.github.com/search/users?q={query}{&page,per_pa
ge,sort,order}'}
```


# 9. Working with a public API

- API: `https://jsonplaceholder.typicode.com/`
- We use the endpoint: `posts`
- To perform the error handling, use `raise_for_status()`
- If no error exists, then `raise_for_status()` gives `None`

In [65]:
```
url = "https://jsonplaceholder.typicode.com/posts"
```

## 9.1

- To make a GET request to retrieve a list of posts

In [66]:
```
try:
        response = requests.get(url)
        response.raise_for_status()
except Exception as e:
        print(e)
else:
        status_code = response.status_code
        print(f"Status Code: {status_code}")
        if status_code == 200:
                print("\nSuccessful GET request!")
                posts = response.json()
                for i in range(3):
                        print(f"\nPost {i + 1}:")
                        print(posts[i])
        else:
                print("Unsuccessful GET request!")
```

```
Status Code: 200

Successful GET request!

Post 1:
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati exc
epturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae conse
quuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum
est autem sunt rem eveniet architecto'}

Post 2:
{'userId': 1, 'id': 2, 'title': 'qui est esse', 'body': 'est rerum tempore vitae
\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis
voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possim
us qui neque nisi nulla'}

Post 3:
{'userId': 1, 'id': 3, 'title': 'ea molestias quasi exercitationem repellat qui i
psa sit aut', 'body': 'et iusto sed quo iure\nvoluptatem occaecati omnis eligendi
aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius
odio et labore et velit aut'}
```

## 9.2

- To make a POST request to submit a new post
- We include the following parameters:
  - title
  - body
  - userId

In [67]:
```
new_post = {
        "title": "Sample Post",
        "body": "This is a sample post",
        "userId": 101
}
```

In [68]:
```python
try:
        response = requests.post(url, data=new_post)
        response.raise_for_status()
except Exception as e:
        print(e)
else:
        status_code = response.status_code
        if status_code == 201:
                print(f"Status Code: {status_code}")
                print("\nSuccessful POST request!")
                post = response.json()
                print("\nPost:")
                print(post)
        else:
                print(f"Status Code: {status_code}")
                print("\nUnsuccessful POST request!")
```

```
Status Code: 201

Successful POST request!

Post:
{'title': 'Sample Post', 'body': 'This is a sample post', 'userId': '101', 'id':
101}
```

In [ ]: